Computers & Security 154 (2025) 104445

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full Length Article

SecRASP: Next generation web application security protection methodology and framework

Chenggang He [a,*], Chris H.Q. Ding [b]

[a] School of Public Safety and Emergency Management, Anhui University of Science and Technology, Hefei, China
[b] School Department of Computer Science and Engineering, University of Texas at Arlington, Arlington, Texas, USA

ARTICLE INFO

Keywords:
Web Application Security Protection
High performance
SecRASP
Quick Block 0-day

ABSTRACT

Currently, Web applications are encountering unprecedented security threats from various types of attacks, such as SQL injection, XSS cross-site scripting attacks, Webshell attacks, and increasingly prominent 0-day security vulnerabilities and other serious threats, the security of Web applications is directly related to the normal order of society and national security and other important aspects. However, the current Web application security protection methods and tools are easy to bypassed by security attacks, have high false positives, and are cumbersome to configure, are unable to protect efficiently against the growing application demands and the increasingly complicated malicious attacks. In this regard, the SecRASP high-performance application security protection method and framework based on RASP are proposed to solve the current Web application security protection problems such as low accuracy, inability to quickly block 0-day security holes, and serious impact on the performance of Web applications and other "choke points". Experimental results show that in terms of security attack protection accuracy, SecRASP's security protection accuracy reaches 100 %, while Baidu OpenRASP's security protection accuracy is only 77.60 %; in terms of the impact on the performance of the protected application, SecRASP's impact on the performance of the application is minimal, compared to the OpenRASP in the CPU utilization rate, memory utilization rate, the performance of the protected application is reduced by 5.2 %.

1. Introduction

With the further development of Internet technology, the current Web applications are facing unprecedented threats of security attacks, such as SQL injection (Mehta et al., 2023), XSS (Kaur et al., 2023) (Cross-site scripting attacks), Webshell attacks (Yu et al., 2024) and increasingly prominent 0day security holes (Guo, 2023) and other serious threats, the security of Web applications is directly related to the normal order of the society and the security of the country and other important aspects. Today's technology for Web application security protection mainly uses WAF (Web Application Firewall),such as ModSecurity (Nasrullayev et al., 2023), in the growing application needs and increasingly complex malicious attacks, WAF is facing more and more challenges, WAF's obvious weaknesses are mainly manifested in the following aspects: (1) Deployment difficulties: most of the WAF is often deployed as on-premise or Rent hosting equipment, and can't be well adapted to the flexible deployment strategy based on the cloud platform. (2) Easy to be bypassed by security attacks, high false positives, cumbersome configuration: today a variety of advanced attacks continue to appear, relying on WAF and other traditional rules of the Web application security protection methods will exist to be frequently bypassed, resulting in a large number of missed reports, making the protection form a false positive. (3) Can't protect against logical vulnerabilities: WAF can't protect against logical vulnerabilities, for the lack of authentication and other logical attacks can't be protected. (4) Can not quickly block 0-day attacks: WAF based on the security rule base of the attack rules to match, such as successfully matching the rule base of the rules, then identified as anomalous and request blocking. (5) Can not support the user to create and add code patches quickly, WAF can't support users to create and add code patches quickly, and often relies on customers to manually modify code after discovering 0-day attacks.

In view of the above shortcomings of Web application security protection, SecRASP high-performance application security protection methodology and framework are proposed based on RASP (Le-Thanh et al., 2023) (Runtime Application self-protection) technology, which is used to solve the "choke point" problems such as low application security

* Corresponding author.
E-mail address: gangzi08@126.com (C. He).

https://doi.org/10.1016/j.cose.2025.104445
Received 26 September 2024; Received in revised form 18 February 2025; Accepted 12 March 2025
Available online 15 March 2025
0167-4048/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

protection accuracy, inability to block 0day security vulnerabilities and serious impact on the performance of the protected Web application. 0-day security holes and serious impact on the performance of the protected Web applications and other "neck" problems. The main contributions of this paper are summarized as follows:

(1) Propose a high-performance application security protection method for web applications, referred to as SecRASP. This method not only inherits the effective code-level security features of RASP but also boasts high accuracy in application security protection. Additionally, it provides rapid protection against zero-day vulnerabilities while maintaining a very low impact on the performance of the protected web applications.
(2) A comprehensive security protection framework based on SecRASP is proposed to enhance security capabilities and enable rapid protection responses. Additionally, a core protection algorithm for web applications is introduced within the SecRASP framework. This integration allows SecRASP to work seamlessly with middleware, ensuring that proactive defense measures are closely aligned with the application itself, creating a cohesive security solution.
(3) By designing multiple sets of experiments on WebGoat (WebGoat), Vulns (Vulns), the industry-famous ranges and JEECMS (JEECMS) applications to verify the high accuracy and performance of SecRASP's security protection. Comparison experiments with Baidu's OpenRASP debugging system (OpenRASP) (OpenRasp is a new type of protection that allows developers to integrate complex security features into application servers even if they have little or no security experience. It is fundamentally different from existing security measures in terms of product design, code implementation and operation. The biggest difference with traditional protection devices is that OpenRASP works directly at the back-end application level and does not rely on rules and blacklists.) show that SecRASP not only has 100 % application security protection accuracy (OpenRASP security protection accuracy is only 77.60 %), but also has a very low impact on the performance of the protected Web applications, reducing CPU utilization by 5.2 % compared to OpenRASP, and memory utilization by 0.41 %. It reduces CPU utilization by 5.2 % and memory utilization by 0.41 % compared to OpenRASP.

The main structure of this paper is organized as follows: section 2 is the analysis of related work; section 3 introduces the SecRASP security protection principle and security protection architecture research; section 4 mainly elaborates the core protection algorithms of SecRASP; section 5 is the experimental validation and analysis; and section 6 concludes this paper with a conclusion and outlook.

2. Related works

In recent years, security issues at the Web application level have become more and more prominent, and are constantly under serious threats such as SQL injection, XSS (cross-site scripting attacks), Webshell attacks, and the increasingly prominent 0day security vulnerabilities. Among them, open-source components such as Apache Log4j2 (Apache Log4j) and Spring (Spring) are frequently disclosed with ultra-high risk security vulnerabilities. In the academic community for Web application security protection technology is widely used based on AST (Hoffman, 2024) (Application Security Testing) research, specifically including DAST (Pagano et al., 2023) (Dynamic Application Security Testing), SAST (Cifuentes et al., 2023) (Static Application Security Testing), and IAST (Pargaonkar, 2023) (Interactive Application Security Testing). However, these techniques have serious drawbacks:

(1) They are only used in the testing and development phase of Web applications, and cannot be used in the production line for real-time application protection;
(2) The DAST false alarm rate is very high often reaching more than 50 %, SAST needs to analyze the source code of the product, which is in the privacy protection and Code confidentiality leakage of hidden dangers, IAST is often tainted with the loss of other problems, often leading to security vulnerabilities in the leakage of the report and false alarms;
(3) Can not quickly and effectively carry out real-time protection of 0day vulnerabilities.

Gartner (RASP) proposed the security protection concept "Runtime application self-protection" referred to as RASP in 2014, and will be listed as a key development trend in the field of future application security. In the past decade, RASP technology is booming, foreign related areas of the famous IT companies are also exploring RASP technology, mainly Micro Focus (Micro Focus), Contrast Security (Contrast Security), Prevoty (Prevoty) and Shadowd (Shadowd), etc., at present, domestic and foreign RASP research more widely explored are Baidu's open source product OpenRASP, but due to its accuracy of Web application security protection, the inability to quickly block 0day security holes and the impact on application performance and other aspects of the problem, has not been widely promoted. At present, RASP-based Web application security related research is basically the use of WAF (web application fire ware) rule matching and hardware combination, such as RASP-based firewalls. Most of these technologies are based on a combination of hardware, from the perspective of analyzing network traffic, without really going deep into the application code level, the disadvantages are as follows:

(1) Cannot accurately locate the specific code location, and cannot perform dynamic tracking of taints;
(2) The high false alarm rate is also a hard problem of this type of technology;
(3) No way to locate the lines of code, methods and parameters, which is not conducive to developers to analyze and repair the vulnerability;
(4) Easily bypassed by security attacks, high false positives, and cumbersome configuration.

3. Research on SecRASP security protection principle and security protection framework

Aiming at the problems of low accuracy, inability to quickly block 0day security holes and serious impact on the performance of the protected application, this paper proposes SecRASP as a new-generation high-performance Web application security protection method and framework, and this subsection mainly elaborates from the principle of SecRASP's security protection and the overall architecture design and other aspects.

3.1. SecRASP security principles

Web application security protection is based on the protection of dynamic access to a variety of information on the protected application, and then all-round, multi-dimensional reorganization and analysis of all kinds of information in the application, so as to more accurately detect common attack instances (such as OWASP Top 10 (OWASP Top) type of security holes, etc.) and logical security (the application's own security) attack examples. SecRASP Security protection method through the application of bytecode (Ding et al., 2023) in the dynamic staking detection "probe" (Agent), real-time access to the context of the application running information, in the application is running, the use of the current context information to protect the application security in real time, such as the SecRASP application security protection shown in

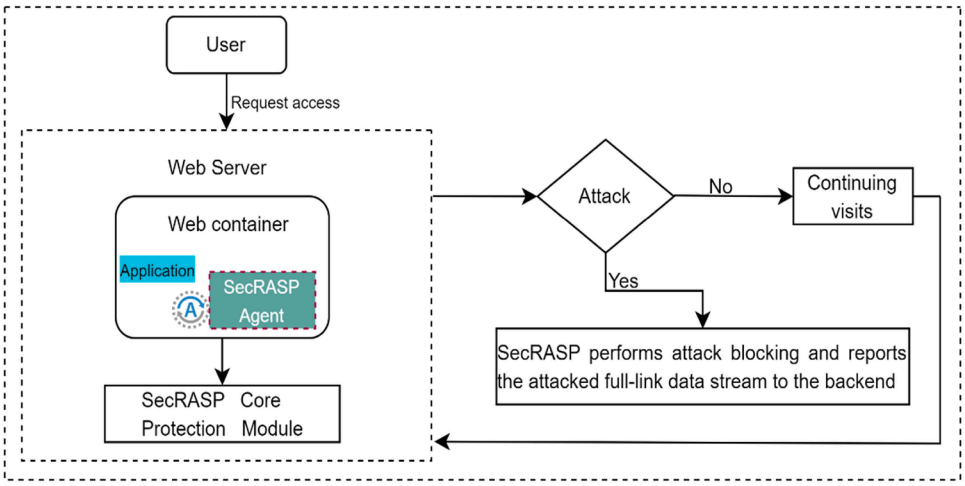

Fig. 1. Schematic diagram of SecRASP application security protection.

Fig. 1. As shown in Fig. 1, SecRASP Application Security Protection is a schematic diagram. SecRASP is designed to encapsulate and protect a specific application rather than a generic network or endpoint layer protection solution. This more purposeful deployment position allows RASP to monitor the inputs, outputs and internal state of the protected application. By deploying RASP, developers are able to identify vulnerabilities within an application. Additionally, RASP solutions are also able to block requests that exploit existing vulnerabilities in the deployed application.SecRASP can inspect legitimate traffic without affecting the normal operation of the application, and SecRASP itself does not store user data or private customer information.

From the SecRASP application security protection schematic shown in Fig. 1 can be seen, SecRASP is directly embedded into the application container and application integration, this feature and WAF are fundamentally different. It is because of this characteristic of SecRASP, SecRASP has an inherent immunity, can circumvent the WAF rules bypassed by a variety of security vulnerabilities fatal weakness.

Staking technology is the basis for SecRASP to obtain application context information. In this paper, we take a Java Web application as an example. Java Instrumentation is a new feature introduced in JDK 1.5 (JDK, Java Develop Kit), which enables developers to build an application-independent Agent to monitor the application running on the JVM by means of a staking detection probe. The basic process of staking a Java application from source code compilation to runtime by SecRASP's Agent is shown in Fig. 2, where the staking process acts at the bytecode level and is triggered when the JVM loads the bytecode program (.class). A Java program from the source code compiled to run the basic process of inserting stakes, where the staking process is acting at the bytecode level, and triggered when the JVM loads the bytecode program (.class). Java agent is a special kind of Java program (jar file), is Instrumentation's client, suitable for appearing in the outer layer of the Java program to evening the Java agent is a special kind of Java program (jar file), is a client of Instrumentation, suitable for appearing in the outer layer of the Java program, in order to the viewer's point of view in the appropriate time to appear, and the ordinary Java program through the main method to start a different, agent is not a program can be started alone, but must be attached to a Java application (JVM), and it is running in the same process, and through the Instrumentation API and the virtual machine to interact with the Java agent is inseparable from Instrumentation. Instrumentation is inextricably linked, and the two

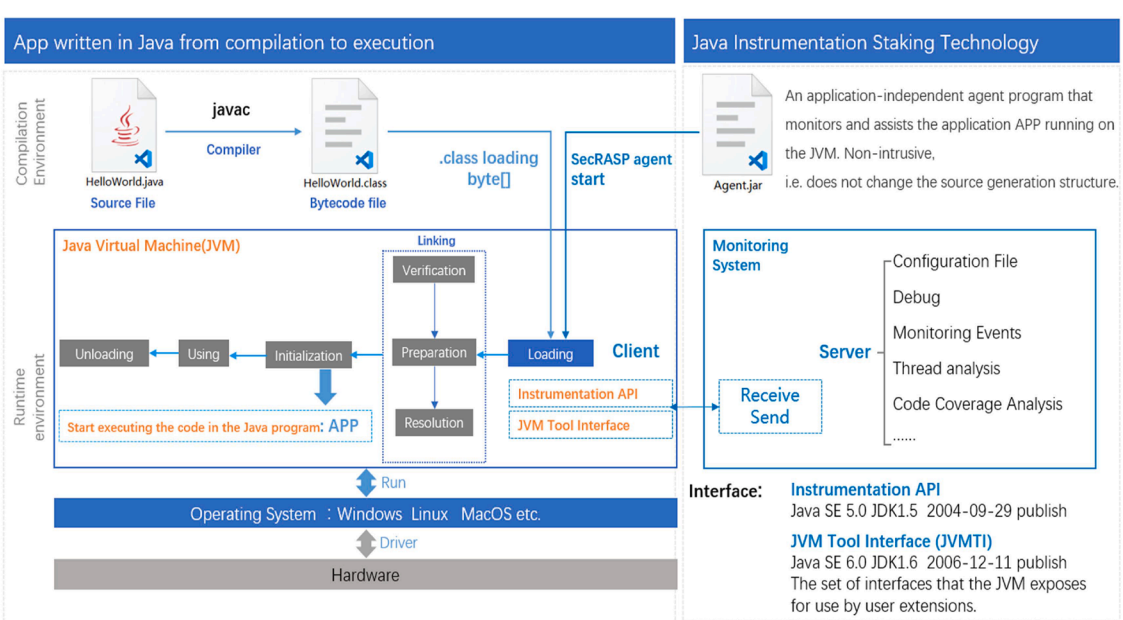

Fig. 2. Schematic diagram of the basic flow of Agent staking in SecRASP.

need to be used together. Because an instance of Instrumentation is injected as a parameter into the Java agent's startup method.

In order to better visualize SecRASP's staking technique, the basic flow of a Java program from source code compilation to running is shown in detail in Figs. 3 and 4, where the staking process occurs at the bytecode level and is triggered when the JVM (Java Virtual Machine) loads the bytecode program (.class). Here agent is a Java program (jar file) that appears in the outer layer of a Java program. Unlike a Java application that is started through the Main method, agent is not a program that can be started separately, but must be attached to a Java application (JVM) and run in the same process that interacts with the VM through the Instrumentation API to interact with the virtual machine. In order to more effectively carry out the application of security protection, SecRASP supports two kinds of agent loading technology applications, 1) cold loading: which can be used for applications in the business scenarios and can be restarted at any time to load agent as shown in Figs. 3, and 2) hot loading: due to some business scenarios are not allowed to restart at any time, for this SecRASP in-depth study of the source code of the JDK on the basis of research and development of runtime loading agent. Load agent at the same time, as shown in Fig. 4.

3.2. The architectural design of SecRASP

In order to carry out application security protection more effectively, the overall architecture design of SecRASP mainly consists of two parts: the SecRASP Agent part and the SecRASP Server part: (1) SecRASP Agent part: SecRASP Agent (probe) is deployed in the protected application container, the user uses the browser or mobile phone to access the application, the generated data flow (request-response) runs through the network, reaches the application server, and the traffic data in the web application is collected at different stages without perception: security detection is completed, and the analysis of the web application is carried out. Users use browsers or mobile terminals to access the application, and the generated data flow (request-response) runs through the network and reaches the application server. Traffic data in the Web application is collected at different stages without perception: security detection is completed, the results of the data are analyzed, and the data are saved; (2) SecRASP Server part: Web application security management personnel can log in to the Web application container through browsers via the Internet or local area network (LAN). Web application security managers can log in to the SecRASP background management system through a browser, via the Internet or LAN, to carry out operations related to Web application security protection, view and verify the results, export reports and other operations. Fig. 5 shows the overall architecture design of SecRASP in detail.

The SecRASP Server is a server deployed in the cloud or locally, and the Agent is a security detection engine deployed in the web server where the application under test is located, which just needs to put the agent and the web application in the same container, e.g., put the agent and the web application in tomcat. The Agent and SecRASP Server communicate via https/http, as shown in Fig. 6, which illustrates the architecture of SecRASP Server and Agent system. The SecRASP Agent part receives various security policies and rules configured on the Server and then collects various contextual information when the application is running. Analyze the collected information to detect the security weaknesses of the program under test and the dependent third-party software, finally, the agent sends the discovered security weaknesses to the Server. At the same time, the SecRASP server part, which firstly provides Agent to download and receive Agent to collect information, provides a visualized interface to display the security protection situation of the tested application in real-time, and show the security status of the product in the form of report, provide a visual configuration for 0-Day attacks, configuration of security policies and rules. SecRasp itself has very good scalable distributed deployment capabilities, before the Web server cluster, set up a load-balancing server, load-balancing device's task is to serve as the entrance to the Web server traffic, select the most appropriate Web server, the client's request forwarded to it for processing, to achieve transparent forwarding from the client to the real server. As a distributed architecture, it is essentially a back-end server as computing resources, and storage resources, encapsulated by a management server into a service to the outside world, the client does not need to care about which machine is really providing the service, in its view, it is as if it is facing a server with nearly unlimited capacity, and essentially, it is the back-end cluster that really provides the service. SecRasp runtime by the Server side and Agent together to complete the application security testing, the application in the running process generates data flow interactions, only need to install the Agent in each single Server. Agent will detect the security risk results reported to the Server side to complete the tool to the application of security testing.The system architecture of SecRASP Server and Agent is shown in Fig. 6.

4. SecRASP core protection algorithm research

Based on the analysis of SecRASP technology principles and overall framework design, SecRASP realizes the seamless integration of proactive defense capabilities into the application and becomes a tightly integrated whole with the application through the close integration of middleware. By listening to key functions of request invocations and combining application context analytics and a powerful attack detection engine, SecRASP can capture and block all kinds of threat attacks that bypass traffic detection, responding to the ubiquitous application vulnerabilities and cyber threats, thus providing dynamic security protection for the whole life cycle of applications.

SecRASP application security protection logic as shown in Fig. 7, first in the Web application container for loading probes, in SecRASP probe loading success, the probe will automatically be the application of key classes and functions for staking. Then SecRASP will analyze the corresponding parameters in the specific function through the interceptor,

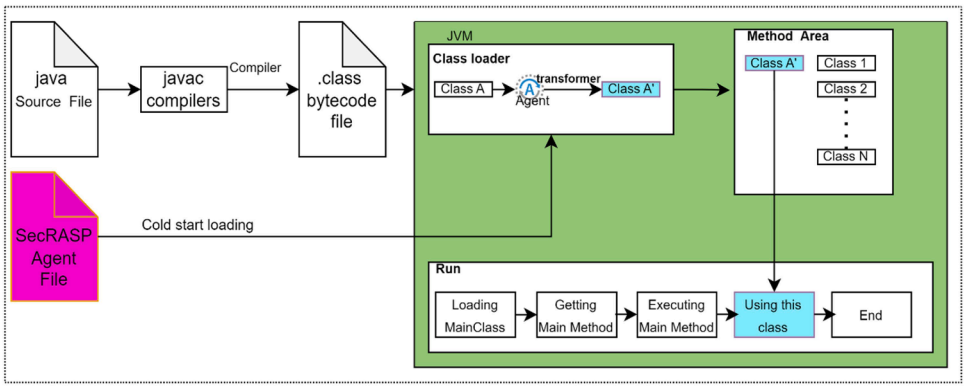

Fig. 3. Schematic diagram of loading SecRASP Agent on startup.

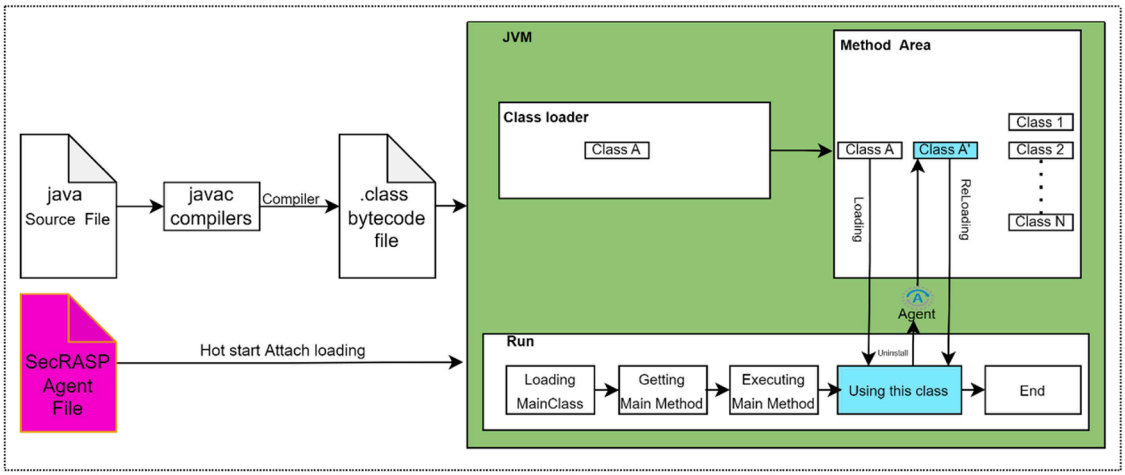

Fig. 4. Schematic diagram of loading SecRASP Agent at runtime.

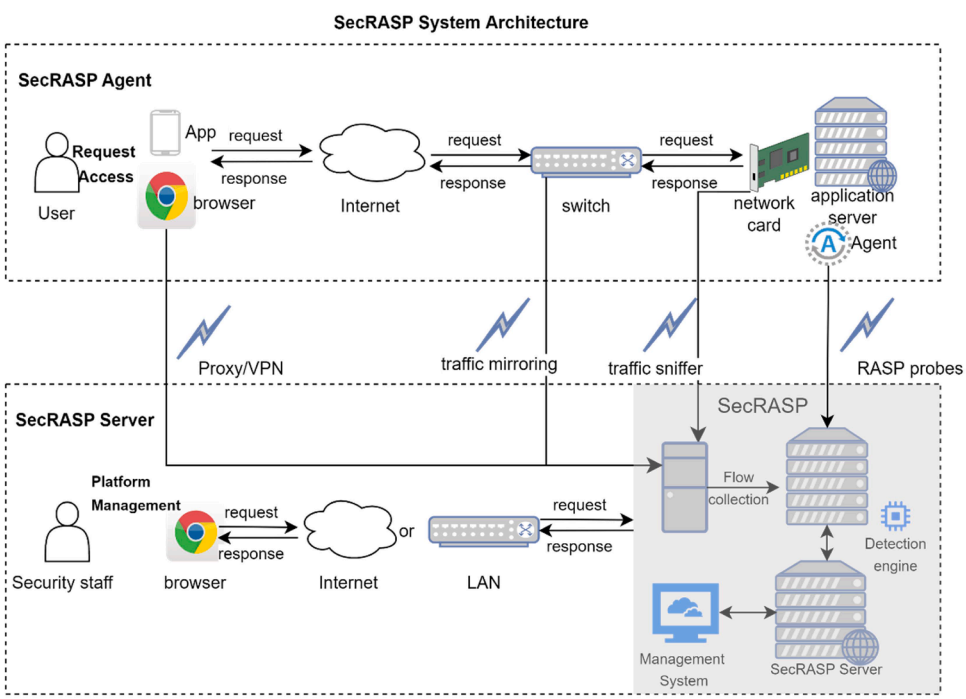

Fig. 5. SecRASP system architecture design diagram.

and carry out the tracking of the whole path of security attacks through the attack event logging module, intercept security attacks through the attack event interception module, and report security attacks through the attack event reporting module, and at the same time, it can also provide targeted and efficient protection through the customization module of the protection rules for the specific security attack characteristics. The protection rule customization module can also be used for targeted and efficient protection against specific security attack characteristics. For SecRASP security attack protection plays an important role is the analysis engine, which through the logical link analysis, vulnerability database in the CVE (Lin et al., 2023), CNNVD (Xiong et al., 2023) and security vulnerability characteristics of the matching, and then through the system logic interface to protect against various types of security attacks. Due to space limitation, this paper focuses on several typical security attack types of SecRASP algorithms in detail, mainly including SecRASP-based SQL injection protection algorithm, SecRASP-based Log4j security vulnerability protection algorithm, SecRASP-based InjectJDBC security vulnerability protection algorithm, SecRASP-based file download security vulnerability protection algorithm, SecRASP-based directory traversal security attack protection algorithm and SecRASP-based StoredXSS security attack protection algorithm.

4.1. SecRASP-based SQL injection protection algorithm

There are a large number of SQL injection attacks in actual web application security attacks, such as CVE-2021-44228 JEECG-Boot SQL Injection Vulnerability (Wong et al., 2023), CVE-2022-26201-Victor CMS SQL Injection Vulnerability (CVE-2022-26201), CNVD-2022-42853-Zendo - Unauthorized SQL Injection Vulnerability (CNVD-2022-42853), etc. Algorithm 1 is widely used in actual Web security protection practice, when the user's input parameters contain some SQL keywords, if the input parameters are not strictly input SQL encoding and validation, it may destroy the original SQL statement

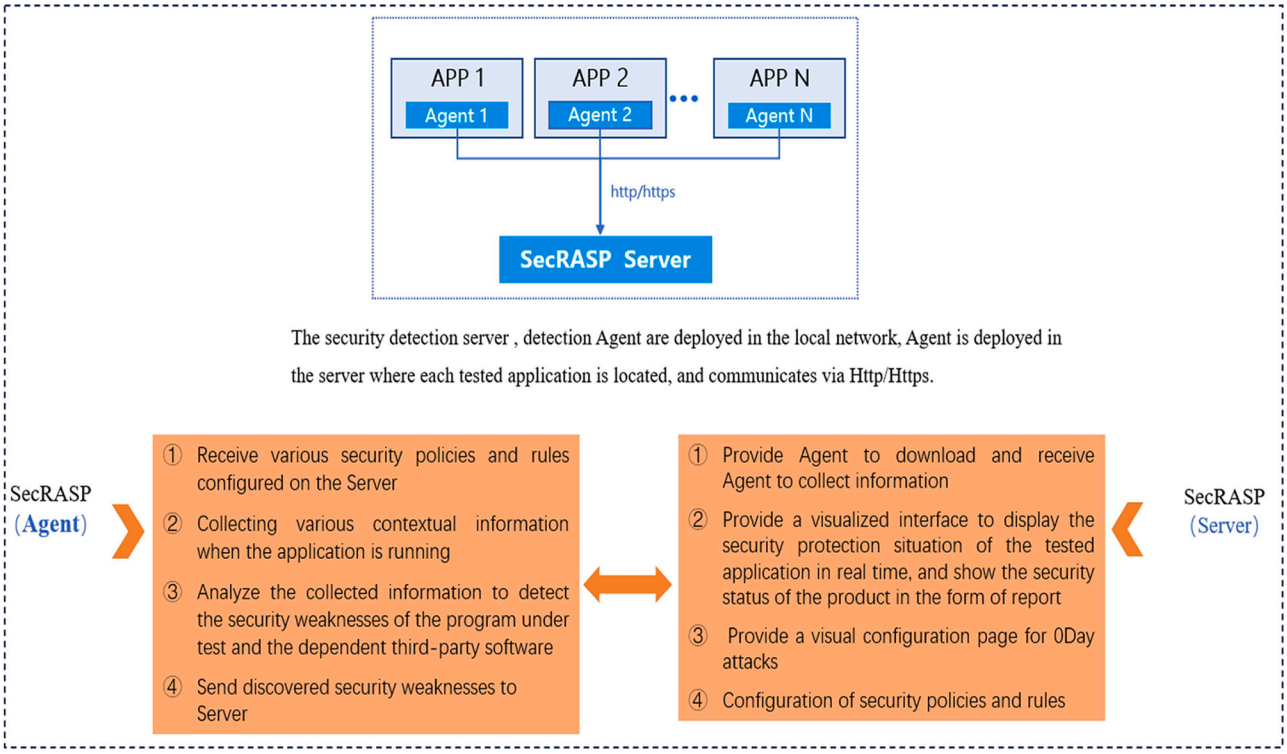

Fig. 6. SecRASP Server and Agent system architecture design diagram.

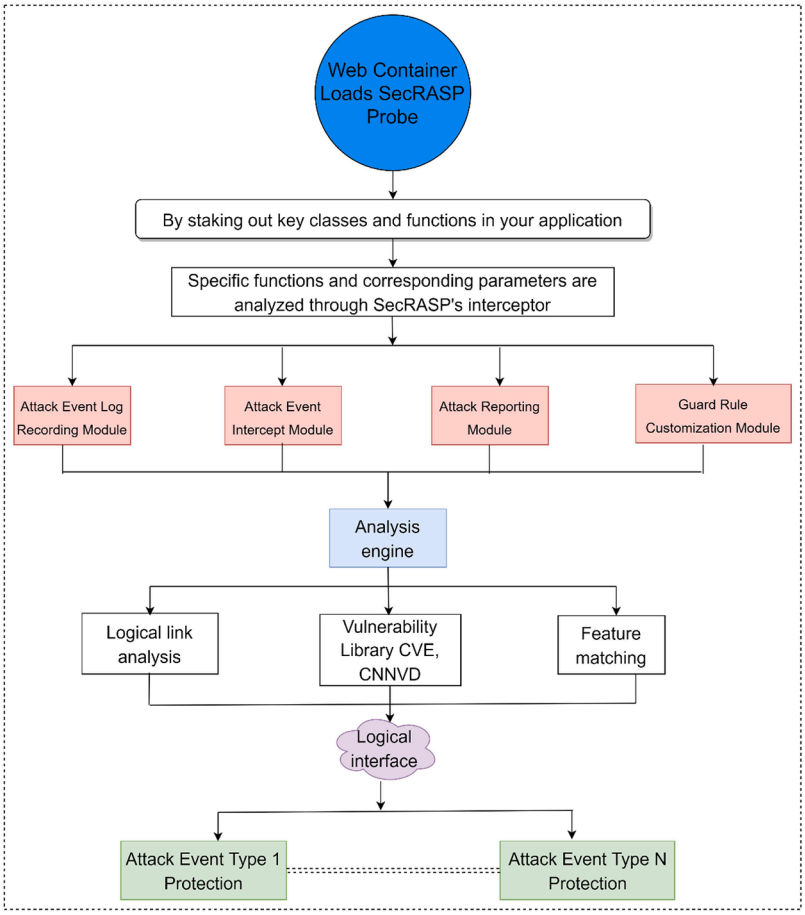

Fig. 7. SecRASP application security protection logic diagram.

Algorithm 1
SecRASP-based SQL injection protection algorithm.

```
Input: SQL Attacks on Web Applications
Output:Successful interception of SQL injection attacks and reporting of related attack
  data to the server.
ProtectSQLInjection(Web Request)
  SetSQLHookFuntion(execute(),executeUpdate(),executeQuery(), append(),
  toString())
Get all web application request parameters:
For (count=0; count≤Length (Requests); count++)
getRequestPara =javax.servlet.ServletRequestWrapper.getParameterValues
  (inputData)
Parameter detection during the request propagation phase
If isFilter(getRequestPara):
Continue;
Else
  CheckParameter = java.lang.StringBuilder.append(getRequestPara)
  CheckParameterStr = java.lang.StringBuilder.toString(CheckParameter)
  Org.hsqldb.jdbc.JDBCStatement.executeQuery(CheckParameterStr)
CheckParameterStr for SQL injection:
  If (CheckParameterStr isContain(tostring(getRequestPara)))
    Return and alert("SQL injection !") //Blocking of SQL Injection Attacks
  Else
    Continue;
EndIf
EndIf
EndFor
```

structure, resulting in SQL injection attacks. Against Algorithm 1 SecRASP based SQL injection protection algorithm to give a concrete example, the user input parameter Bob'or '1′='1, in the propagation phase of the request through the detection of pre-piled functions for parameter tracking, java .lang.StringBuilder.append(Bob' or '1′='1) . This is expressed in the database SQL statement as Select * From user_data where user_name= Bob' or '1′='1, after the request parameter has been passed through java.lang.StringBuilder.toString(), then the SQL statement in the database will be as follows Select * From user_data where user_name='Bob' or '1′='1′, if at this point there is no blocking of the if the SQL execution is not blocked at this point, the SQL statement will be executed in Org.hsqldb.jdbc.JDBCStatement.executeQuery (Select * From user_data where user_name='Bob' or '1′='1′) is executed, which triggers the SQL attack. Through SecRASP security vulnerability protection module, SQL injection exists in the output phase of the final request parameters, and the Web application security attack is intercepted and related data information is reported to the Server, thus protecting the Web application.

Algorithm 2
SecRASP-based Log4j security vulnerability protection algorithm.

```
Input: Web application traffic
Output:Successful interception of Log4j security attacks and reporting of related
  attack data to the server
ProtectLog4JInjection(Web Request)
SetHookFuntion(org.apache.logging.log4j.core.net.JndiManager. Lookup())
Get all loaded classes:
Class[] classes = Instrumention.getAllLoadClasses();
For(count=0; count≤Length(classes); count++)
  Class oneClass = classes[i];
Determine if the retrieved class is JndiManager class:
If NotJndiManager.class.isAssignableFrom(oneClass):
  Continue;
Else
  If isExist(getMethod(ladp, org.apache.logging.log4j.core.net.JndiManager))
    Return and alert("Log4j injection!")
  Else
    Continue;
  EndIf
  EndIf
EndFor
```

4.2. Log4j security vulnerability protection algorithm based on SecRASP

In the actual practice of Web security protection, Algorithm 2 has an important role, Log4j2 (Ferreira et al., 2023) for developers are no strangers, mainly used to record the project logging tools, to the file or console to print the debugging information of the code, is the development and production environment of the gods, can help developers pinpoint the problem, but also includes an understanding of the system's operational status. In 2023 researchers discovered a serious security vulnerability CVE-2021-44228 (Remote Code Execution Vulnerability) (Wong et al., 2023) in Log4j2, caused by the lookup feature in Log4j2, which allows developers to remotely read configurations over the protocol. There is no strict judgment in this process, leading to a serious risk that code that is harmful can be downloaded remotely and executed, which leads to the server being implanted with a virus that can be used for other purposes by the attacker. Because this vulnerability is quite serious, it will lead to the server being completely controlled by the attacker, and turned into a zombie machine, in addition to log4j2 this logging tool is used in a wide range of projects, many open-source projects use this logging framework, such as Kafka (Peddireddy, 2023), Solr (Solr, 2023), etc., so from the vulnerability itself, as well as the extent of the harm, the security vulnerability is also known as the "nuclear bomb" vulnerability. The core algorithm of SecRASP can be used to effectively protect Log4j2 against this 0day vulnerability in an unaware manner.

4.3. SecRASP-based security vulnerability protection algorithm for InjectJDBC

Recently, the famous VMware company has been missed in its product VMware Workspace ONE Access (Lombard, 2023) OAuth2 ACS framework there is CVE-2022-22955 (CVE-2022-22955) authentication bypass security attack vulnerability, the attacker can use the vulnerability to obtain the activation code of the OAuth2 client, and then through the activation of the OAuth2 client to bypass authentication. OAuth2 client to bypass authentication, after analyzing CVE-2022-22957 (CVE-2022-22957) belongs to the remote code execution caused by JDBC injection, the attacker with administrative access to construct a malicious JDBC URL through the controllable parameters to trigger the deserialization, so as to execute arbitrary

Algorithm 3
SecRASP-based Security Vulnerability Protection Algorithm for InjectJDBC.

```
Input: Web application traffic
Output:Successfully intercept the InjectJDBC security attack and report the related
  attack data information to the server
ProtectInjectionJDBC(Web Request)
Get all loaded classes:
Class [] classes = Instrumention.getAllLoadClasses();
For (count=0; count≤Length(classes); count++)
  Class oneClass = classes[i];
Determine if the fetched class is a Driver class:
If ! Driver.class.isAssignableFrom(oneClass):
  Continue;
Else
  If isExist(getMethod(connect, java.sql.Driver))
Get the parameter string of the connect() method through java reflection:
  paraString = getParameterFrom(connect());
Detecting whether to attack scripts:
If (isAttack(paraString)
    Return and alert("JDBC Attack!")
  Else
    Continue;
  Else
    Continue;
  EndIf
  EndIf
EndIf
EndFor
```

commands to obtain system privileges. Through the built-in protection rules of Algorithm 3 in this paper can InjectJDBC can be automatically in the java.sql.Driver subclass of the connect method of the senseless insertion of stakes, so as to efficiently carry out the security protection of InjectJDBC.

4.4. SecRASP based algorithm for file download security vulnerability protection

In a Web application often because of certain business needs, need to provide file (attachment) download function. However, if there is no restriction on the downloaded files, and the files are downloaded directly through absolute paths, then malicious users can take advantage of this way to download the server's sensitive files, and carry out further threats and attacks on the server. Recently announced CNVD-2023-11441 (CNVD-2023-11441) high-risk security vulnerability, the vulnerability is caused by the lack of external input data validation of the parameter download of the source file downloadFiles, the attacker can use this vulnerability to cause arbitrary file downloads, which can be seen in the great harm of this security vulnerability.SecRASP can obtain various runtime context information of the application through Algorithm 4, so that the application has the ability of self-protection, which can quickly and efficiently carry out the real-time protection of the file download security vulnerability.

4.5. SecRASP-based algorithm for protection against directory traversal security attacks

IBM has recently been alerted to an ultra-critical CNVD-2023-70071 (CNVD-2023-70071) directory traversal vulnerability in its product IBM Security Directory Server version 7.2.0, which stems from a lack of validity checking of a program's paths when processing directory requests, and which can be exploited by an attacker to send a specially crafted URL request to view or write an arbitrary file on a system. arbitrary files on the system. A directory traversal vulnerability means that the application directly uses user input data to manipulate a file, and a malicious user can construct a directory traversal request in the input parameter similar to "... /... /... /..." file path in the input parameter to complete the directory jump, so as to achieve the purpose of reading or writing files in any directory of the operating system. SecRASP's Algorithm 5 can obtain various runtime context information of the application, so that the application has the ability to self-protect, and thus quickly and efficiently carry out directory traversal security attacks security vulnerabilities in real-time protection.

Algorithm 4
SecRASP based file download security vulnerability protection algorithm.

```
Input: Web application traffic
Output: Successful interception of file download security attacks and reporting of
  related attack data information to the server
        ProtectFileDownloadInjection(Web Request)
        Get all loaded classes:
        Class [] classes = Instrumention.getAllLoadClasses();
        For (count=0; count≤Length(classes); count++)
            Class oneClass = classes[i];
        Determine if the retrieved class is a File class:
            If ! java.io.File.class.isAssignableFrom(oneClass):
            Continue;
            Else
            If isExist(getAbsolutePath, java.io.File))
        Get the parameter string of the getAbsolutePath () method via java
        reflection.
            paraString = getParameterFrom(getAbsolutePath ());
        Detecting whether to attack scripts
            If (isAttack(paraString))
            Return and alert("File Download injection!")
              Else
              Continue;
          Else
        Continue;
      EndIf
      EndIf
    EndIf
  EndFor
```

4.6. SecRASP based algorithm for StoredXSS security attack protection

Adobe has recently been alerted to a high-risk CNVD-2022-79410 StoredXSS (CNVD-2022-79410) security vulnerability in its product Adobe Experience Manager (AEM), which can be exploited by an attacker to execute malicious JavaScript content in the context of the victim's browser. The XSS vulnerability can be exploited by an attacker to corrupt the context structure of an output point to inject arbitrary executable code, steal sensitive information such as usernames and passphrases (which may be contained in a cookie) from other users' sessions, or perform a mounted attack on a user by inserting malicious code. SecRASP's Algorithm 6 can obtain various runtime contextual information of the application, so that the application has the ability to self-protect, and thus quickly and efficiently carry out real-time protection against StoredXSS security attack vulnerabilities.

5. Experimental results and analysis

5.1. Experimental environment and sample set of security attacks

In this paper, in order to verify the ability of SecRASP to protect the security of Web applications, the experiment selected the Baidu Vulns range and Webgoat range recognized by academia and industry as a security attack sample set, in which the Vulns range includes directory traversal vulnerabilities, file download vulnerabilities, command line execution vulnerabilities, arbitrary file write vulnerabilities, sensitive information leakage vulnerabilities, XXE vulnerabilities, Deserialization vulnerability, SSRF vulnerability, SQLi vulnerability, XSS vulnerability, Loadlibrary vulnerability, arbitrary file deletion vulnerability, and other 12 categories of 31 attack samples.Webgoat6.0 includes XSS attacks, Injection Flaws attacks, Access Control Flaws attacks, and other 3 categories of 15 security attack samples. Webgoat 8.0 includes three major categories of attack samples such as SQL Injection attack, XXE attack and XSS attack. Meanwhile, in order to verify the impact of SecRASP on the performance of the protected Web applications, JEECMS applications are used for security attack performance testing. At the same time, we compare SecRASP with Baidu's OpenRASP product in terms of security

Algorithm 5
SecRASP-based directory traversal security attack protection algorithm.

```
Input: Web application traffic
Output: Successful interception of directory traversal security attacks and reporting of
  related attack data information to the Server
  ProtectDirectoryTraversalInjection(Web Request)
  Get all the parameters for a file operation:
  For (count=0; count≤Length (Request); count++)
    getFilePara = org.apache.catalina.connector.Request.getParameter
  (fileName)
  Parameter detection during the request propagation phase:
    If isInValidFileName(getFilePara):
      Break;
  Else
    CheckParameter = java.lang.StringBuilder.append(getFilePara)
    CheckParameterStr = java.lang.StringBuilder.toString(CheckParameter)
    java.io.File(CheckParameterStr)
    CheckParameterStr for directory traversal security injection
    If (isAttack(CheckParameterStr)
      Return and alert("Directory Traversal injection!")
    Else
      Continue;
    EndIf
    EndIf
  EndFor
```

Algorithm 6
SecRASP-based StoredXSS security attack protection algorithm.

```
Input: Web application traffic
Output: Successful interception of stored XSS security attacks and reporting of related
  attack data to the server
    ProtectStoredXSSInjection(Web Request)
    SetHookFuntion(javax.servlet.ServletRequestWrapper(inputData))
    Get all the parameters for a file operation:
    For (count=0; count≤Length (Requests); count++)
      getRequestPara = javax.servlet.ServletRequestWrapper(inputData)
      Parameter checking on input data:
      If isInValidFileName(getRequestPara):
        Break;
      Else
        CheckParameter = java.lang.StringBuilder.append(getRequestPara)
      CheckParameterStr = java.lang.StringBuilder.toString(CheckParameter)
      java.io.PrintWriter.write(CheckParameterStr)
      Checks CheckParameterStr for XSS injection:
      If isAttack(CheckParameterStr):
      Return and alert ("StoredXSS injection!")
        Else
        Continue;
      EndIf
      EndIf
    EndFor
```

attack protection capability and impact on the performance of the protected Web applications.

5.2. Comparison of accuracy aspects of security attack protection capabilities

In order to further verify the security attack protection capability of SecRASP, this section compares OpenRASP with Baidu's OpenRASP on Vulns range, Webgoat 6.0 range and Webgoat 8.0 range, and the specific verification results can be found in Tables 1-4. Finally, through the verification on the ranges recognized by both academia and the industry, we can conclude that The accuracy of OpenRASP's security protection is 77.60 %, while the accuracy of SecRASP's security protection reaches 100 %.

5.3. Comparison in terms of security attack protection performance

In order to further verify the impact of SecRASP on the performance of the protected Web application, this paper uses the JEECMS application for security performance testing. The specific performance testing environment is described in Section 5.3.1, which can be found in Tables 5-7. The detailed performance testing methodology is described in Section 5.3.2. Meanwhile, the impact of SecRASP on the performance of the protected Web application is compared with Baidu's OpenRASP product, and the final performance comparison results are revealed in Section 5.3.3.

5.3.1. Test environment

5.3.2. Test methods

Performance testing (He and Ding, 2024, He et al., 2022, He and Ding, 2025) uses the JMeter tool to record security attack scripts on the test application, sets up different thread groups to execute the scripts, and saves the aggregated report when the script execution is complete. Dstat command is used on the server side to count CPU and memory usage information. The test is divided into three groups of comparison experiments, the first group of experiments in the application under test without adding load probe for testing, the second group of experiments in the application under test to add SecRASP probe for testing, the third group of experiments in the application under test to add OpenRASP probe for testing, the detailed info can be found in Groups 1-3.

Table 1
Comparison of security attack protection capability of SecRASP vs OpenRASP on vulns range.

Number	User case	OpenRasp	SecRASP
1	Iterate through directories using File.listFiles (with path stitching)	P	P
2	Arbitrary File Download/Read Vulnerability (Path Splicing)	P	P
3	Command Execution Backdoor - No Showback	P	P
4	Command Execution Backdoor – with Flashback	P	P
5	Write to any file	P	P
6	Check if sensitive information such as IDs and bank cards are leaked in the response	P	P
7	Reading system files via XXE	P	P
8	Reading system files via XXE - dom4j	P	P
9	Reading system files via XXE - jdom	P	P
10	Reading system files via XXE -sax	P	P
11	Reading system files via XXE - stax	P	P
12	Arbitrary File Upload Vulnerability - commons.io Approach	P	P
13	Transformer Deserialization	P	P
14	JSTL import arbitrary file containing SSRF	P	P
15	SSRF - commons.httpclient approach	P	P
16	SSRF - HttpClient approach	P	P
17	SSRF - URL.openConnection approach	P	P
18	SSRF – OKHTTP approach	P	P
19	SSRF - OKHTTP3 approach	P	P
20	SQLi - MySQL executeQuery approach	P	P
21	SQLi - MySQL 8.X+executeQuery approach	P	P
22	SQLi - MySQL 8.X+preparedQuery approach	P	P
23	SQLi - HSQLDB executeQuery approach	P	P
24	SQLi - Mybatis Mysql JDBC	P	P
25	SQLi - Hibernate Mysql JDBC	P	P
26	SQLi - JDBC multipart approach	P	P
27	XSS - 017-Reflective XSS	P	P
28	Loadlibrary	P	P
29	Deletion of arbitrary documents	P	P
30	RandomAccessFile file reading and writing	P	P
31	NIO file operations	P	P

Table 2
Comparison of security attack protection capability of SecRASP vs OpenRASP on webgoatV6 range.

Module	Number	OpenRasp	SecRASP
Cross-Site Scripting (XSS)	Stage1: Stored XSS	P	P
	Stage2: Block Stored XSS using input Validation	P	P
	Stage3: Stored XSS Revisited	P	P
	Stage4: Block Stored XSS using Output Encoding	P	P
	Stage5: Reflected XSS	P	P
	Stage6: Block Reflected XSS	P	P
Injection Flaws	Stage1: String SQL Injection	Î	P
	Stage2: Parameterized Quuery #1	Î	P
	Stage3: Numeric SQL Injection	Î	P
	Stage4: Parameterized Query #2	Î	P
	String SQL Injection	Î	P
Access Control Flaws	Stage1:Bypass Business Layer Access Control	Î	P
	Stage2: Add Business Layer Access Control	Î	P
	Stage3: Bypass Data Layer Access Control	Î	P
	Stage4: Add Data Layer Access Control	Î	P

5.3.3. Test results and analysis

This can be derived from the data in Table 8 SecRASP vs OpenRASP Performance Test Results Comparison Table:

1) **When the probe is not loaded**, the time required for the requests gradually increases, and the average number of request processing

Table 3
Comparison of security attack protection capabilities of SecRASP vs OpenRASP on webgoatV8 range.

Attack Type	OpenRasp	SecRASP
SQL Injection	P	P
XXE	Î	P
XSS	Î	P

Table 4
Summary of security attack protection capability comparison of SecRASP vs OpenRASP.

Safety Protection Indicators	OpenRASP	SecRASP
Number of security guards	38/49	49/49
Security protection ratio	77.60 %	**100** %

Table 5
Performance test environment hardware configuration table.

CPU	HardDisk	Memory
4 Intel(R) Xeon(R) E3-1226 v3 @ 3.30GHz	75.2G	8G

Table 6
Performance test environment software configuration table.

Operating system	Middleware	DB	Test application
Linux	Tomcat 8.5.66	MySQL 5.7.34	JEECMS

Table 7
Performance testing tool list.

Tool	Tool Description
JMeter (Siahaan and Wijaya, 2024)	Stress Testing Tools
Dstat (Dstat)	Performance monitoring tools

transactions per second first shows an upward trend, and then tends to stabilize, the duration field represents the time consumed by all requests, when the number of thread groups is 15, the average number of request transactions per second reaches a maximum of 267, the corresponding CPU utilization is 82.4 %, and the average memory utilization is 15.99 %. The overall average number of requests per second for unloaded probes is 246, with an average CPU utilization of 72.95 % and an average memory utilization of 15.96 %.

2) **When loading the SecRASP probe**, the time required for the requests gradually increases, and the average number of request processing transactions per second first shows an upward trend, and then tends to stabilize, when the number of thread groups is 15, the average number of request transactions per second reaches a maximum of 256, corresponding to 87.55 % CPU utilization, and 16.96 % memory utilization. The overall average number of requests per second for unloaded probes is 198, with an average CPU utilization of 76.92 % and an average memory utilization of 16.41 %.
3) **When the OpenRASP probe is loaded,** the time required for 90 % of the requests gradually increases, and the average number of request processing transactions per second first shows an upward trend, and then tends to stabilize, and when the number of thread groups is 15, the average number of request transactions per second reaches a maximum of 145, corresponding to a CPU utilization rate of 86.47 %, and a memory utilization rate of 16.84 %. The overall average number of requests per second for unloaded probes was 241, with an average CPU utilization of 82.12 % and an average memory utilization of 16.82 %.

Based on the analysis of the test results, it can be seen that adding the probe increases the CPU usage and memory usage relative to not adding the probe. Loading SecRASP probe increases CPU utilization by 3.98 % and memory utilization by 0.45 %, comparing with OpenRASP probe's 9.18 % CPU utilization and 0.86 % memory utilization, so it is concluded that **SecRASP occupies less system resources, comparing with OpenRASP in terms of CPU utilization decreased by 5.2 % and in terms of Memory utilization is reduced by 0.41** %; Loading the SecRASP probe increases Average transaction response time (ATRT) by 2.25 ms and decreases Average number of transactions per second (TPS) by 5.23 times per second. In comparison, the OpenRASP probe increases ATRT by 18.11 ms and decreases TPS by 47.77 times per second. Therefore, **it can be concluded that SecRASP has lower response time and network latency. The experimental results indicate that SecRASP has a lesser impact on application performance and can support the normal operation of the application.**

6. Conclusion and outlook

Traditional Web application security protection methods and tools are easy to be bypassed by security attacks, high false positives, cumbersome configurations, and are unable to efficiently protect against growing application demands and increasingly complex malicious

Group 1
No probes added.

Step 1: Deploy the application JEECMS into a Tomcat container on a Linux server.
Step 2: Start Tomcat.
Step 3: Start JMeter, click start to start the execution of the preset security attack jmx test script, script execution is complete, save the dstat command resource statistics.
Step 4: While the jxm script is executing, use the dstat command on the Linux server to collect data. When the jmx script finishes executing, stop the dstat command to save the resource usage statistics.
Step 5: Repeat steps 3-4, adding a set of tests for every 5 thread groups from 0-20.
Step 6: Once the tests are completed, organize the data.

Group 2
Installation of the SecRASP probe.

Step 1: Deploy the application JEECMS into a Tomcat container on a Linux server.

Step 2: Start Tomcat.
Step 3: Start JMeter, click start to start the execution of the preset security attack jmx test script, script execution is complete, save the dstat command resource statistics.
Step 4: While the jxm script is executing, use the dstat command on the Linux server to collect data. When the jmx script finishes executing, stop the dstat command to save the resource usage statistics.
Step 5: Repeat steps 3-4, adding a set of tests for every 5 thread groups from 0-20.
Step 6: Once the tests are completed, organize the data.

Group 3
Install baidu OpenRASP probe.

Step 1: Deploy application JEECMS into Tomcat on Linux server.
Step 2: Download the OpenRasp probe rasp.jar and add the probe rasp.jar script to tomcat/bin/catalina.sh.
Step 3: Start Tomcat.
Step 4: Start JMeter, click start to start the execution of the preset security attack jmx test script, script execution is complete, save the dstat command resource statistics.
Step 5: While the jxm script is executing, use the dstat command on the Linux server to collect data. When the jmx script finishes executing, stop the dstat command to save the resource usage statistics.
Step 6: Repeat steps 4-5, adding a set of tests for every 5 thread groups from 0-20.
Step 7: When the tests are complete, organize the data.

Table 8
SecRASP vs OpenRASP performance test results comparison table.

NO	Duration (sec)	Number of Concurrent Requests	Number of requests executed	Average transaction response time (ms)	Maximum Transaction Response Time (ms)	Average number of transactions per second (times/s)	Transaction success rate	CPU usage	Memory Usage
JEECMS									
1	26	5	5425	23	597	202.99	100.00 %	48.66 %	15.84 %
2	43	10	10850	38	652	250.14	100.00 %	76.56 %	15.90 %
3	61	15	16275	54	1083	267.25	100.00 %	82.40 %	15.99 %
4	82	20	21700	74	992	264.62	100.00 %	84.16 %	16.11 %
Average	53	12.5	13563	47.25	831	246.25	100.00 %	72.95 %	15.96 %
JEECMS+SecRASP									
1	41	5	5425	26	501	180.11	100.00 %	61.78 %	16.68 %
2	76	10	10850	37	914	257.58	100.00 %	87.36 %	16.82 %
3	125	15	16275	57	1120	256.7	100.00 %	87.55 %	16.96 %
4	160	20	21700	72	923	269.7	100.00 %	90.87 %	17.39 %
Average	100.5	12.5	13563	**48**	**864.5**	**241.02**	100.00 %	**76.92** %	**16.41** %
JEECMS+OpenRASP									
1	30	5	5425	37	963	130.58	100.00 %	73.42 %	16.67 %
2	41	10	10850	68	1616	143.43	100.00 %	81.16 %	16.77 %
3	64	15	16275	102	1216	145.01	100.00 %	86.47 %	16.84 %
4	81	20	21700	145	1681	136.06	100.00 %	87.44 %	17.00 %
Average	54	12.5	13563	65.36	1070.11	198.48	100.00 %	82.12 %	16.82 %

Adding probes increases the time required for the requests and decreases the average number of transactions processed per request per second compared to not adding probes, due to the fact that the loaded probes have to detect each request, which leads to a decrease in the number of transactions processed per unit of time. The maximum, minimum and average values of CPU, memory utilization, ATRT (Average transaction response time (ms)) and TPS (Average number of transactions per second (times/s)) of the server with and without probes are shown in the Table 9 and Table 10.

Table 9
SecRASP vs OpenRASP performance Test CPU and memory metrics comparison table.

Condition	No probe		SecRASP Probe		OpenRASP Probe		SecRASP Probe vs No Probe		OpenRASP Probe vs No Probe	
Indicators	**CPU**	**Memory**	**CPU**	**Memory**	**CPU**	**Memory**	**CPU**	**Memory**	**CPU**	**Memory**
Minimum	48.66 %	15.84 %	61.78 %	16.68 %	73.42 %	16.67 %	6.71 %	1.28 %	3.28 %	0.89 %
Maximum	84.16 %	16.11 %	90.87 %	17.39 %	87.44 %	17.00 %	13.12 %	0.84 %	24.76 %	0.83 %
Average	72.95 %	15.96 %	76.92 %	16.41 %	82.12 %	16.82 %	**3.98** %	**0.45** %	9.18 %	0.86 %

attacks. There is a fatal drawback about OpenRASP, OpenRASP's Engine class, it does not do the detection logic and business application isolation, because it is loaded by the extension class loader, which will possibly lead to some class conflict problems. For example, the business application is loaded by the application class loader, which introduces X.jar, and Y.jar. In X.jar there is the class com.nnn.X with the public modifier, which inherits the class com.nnn.Y with a protected modifier in Y.jar, and it happens that in Engine's engine.jar it also depends on Y.jar. Then, according to the two-parent delegation mechanism, class com.yyyy.X will be loaded by its mother class loader, which is the extension class loader, and this will eventually result in an exception being thrown because a.jar and b.jar are not loaded by the same class loader. To deal with the shortcomings of OpenRASP, SecRASP not only tries to avoid using jar references in Engine that will cause conflicts but also has a way

Table 10
SecRASP vs OpenRASP performance test ATRT and TPS metrics comparison table.

Condition	No probe		SecRASP Probe		OpenRASP Probe		SecRASP Probe vs No Probe		OpenRASP Probe vs No Probe	
Indicators	**ATRT**	**TPS**	**ATRT**	**TPS**	**ATRT**	**TPS**	**ATRT**	**TPS**	**ATRT**	**TPS**
Minimum	23.00	202.99	26.00	180.11	37.00	130.58	3.00	-22.88	14.00	-72.41
Maximum	74.00	267.25	72.00	269.70	145.00	145.01	-2.00	2.45	71.00	-122.24
Average	47.25	246.25	48.00	241.02	65.36	198.48	**2.25**	**-5.23**	18.11	-47.77

* ATRT stands for Average transaction response time (ms), TPS stands for Average number of transactions per second (times/s).

to isolate its loader, that is, to load Engine through an application class loader, instead of using an extension class loader to load. Specific implementation, SecRASP in the hook insert code, in advance of the method to call the method to reflect the load, and then hosted in the startup class loader to load the Agent, SecRASP will be able to do Engine and business application loader isolation, but also in the isolation of the realization of the same time, to achieve in any class of any method to call to the Engine's classes and methods can be called from any class and any method at the same time.

First of all, this paper proposes SecRASP high-performance application security protection methodology and framework based on RASP, which is used to solve the current industry Web application security protection accuracy is low, cannot quickly block 0day security holes and serious impact on the performance of Web applications, etc. SecRASP not only inherits the original code level of RASP, but also provides the most effective security protection for Web applications. SecRASP not only inherits the original code-level effective security protection features of RASP, but also has the advantages of high accuracy of application security protection, fast protection against 0day security vulnerabilities, and extremely low impact on the performance of the protected Web applications. Then this paper elaborates on the application core protection algorithm based on SecRASP security protection methodology and framework, which enables SecRASP to realize the seamless integration of active defense capabilities into applications and become a close whole with applications through the close integration with middleware. Finally, multiple experiments are designed to verify the effectiveness of SecRASP on WebGoat, Vulns, and JEECMS. By comparing the experiments with Baidu's OpenRASP, the experiments show that SecRASP not only has 100 % application security protection accuracy, but also has a very low impact on the performance of the protected Web applications, reducing 5.2 % in CPU utilization and 0.41 % in memory utilization than OpenRASP. In the subsequent research work, in response to the current rapid mutation of security vulnerabilities, SecRASP will further cover more types of security vulnerabilities protection, and try to combine with the large model to further study the security protection of Web applications.

Availability of data and materials

The datasets used in this article are freely available on Internet. Detailed experimental results can be obtained from the author if necessary.

Funding

This research was funded by Anhui Postdoctoral Scientific Research Program Foundation (No.2024C931), Scientific Research Foundation for High-level Talents of Anhui University of Science and Technology (No.2023yjrc120), Anhui Quality Engineering Project (No.2023cyts013), Anhui University of Technology Postgraduate Education Teaching Research Project(No. 2024yjy009), Natural Science Foundation of China (No.61976004,61572030), NSFC Key Project of International (Regional) Cooperation and Exchanges (No.61860206004).

CRediT authorship contribution statement

Chenggang He: Writing – review & editing, Writing – original draft, Validation, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Chris H.Q. Ding:** Validation, Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The authors would like to thank the editors and anonymous referees.

Data availability

Data will be made available on request.

References

Mehta, D., Suhagiya, H., Gandhi, H., Jha, M., Kanani, P., Kore, A., 2023. SQLIML: a comprehensive analysis for SQL injection detection using multiple supervised and unsupervised learning schemes. SN Comput. Sci. 4 (3), 281.
Kaur, J., Garg, U., Bathla, G., 2023. Detection of cross-site scripting (XSS) attacks using machine learning techniques: a review. Artif. Intell. Rev. 56 (11), 12725–12769.
Yu, Xiaobo, et al., 2024. TridentShell: an enhanced covert and scalable backdoor injection attack on web applications. J. Netw. Comput. Appl. 223, 103823.
Guo, Yang., 2023. A review of machine learning-based zero-day attack detection: challenges and future directions. Comput. Commun. 198, 175–185.
Nasrullayev, N., Muminova, S., Istamovich, D.K., Boltaeva, M., 2023. Providing IoT Security in Industry 4.0 using Web Application Firewall. In: In 2023 4th International Conference on Electronics and Sustainable Communication Systems (ICESC). IEEE, pp. 1788–1792.
Le-Thanh, Phuc, Le-Anh, Tuan, Le-Trung, Quan, 2023. Research and development of a smart solution for runtime web application self-protection. In: Proceedings of the 12th International Symposium on Information and Communication Technology, pp. 304–311.
WebGoat, https://github.com/WebGoat/WebGoat.
Baidu Vulns, https://packages.baidu.com/app/openrasp/testcases/vulns.war.
JEECMS, https://www.jeecms.com/.
OpenRASP, https://github.com/baidu/openrasp.
Apache Log4j, https://logging.apache.org/log4j/2.x/.
Spring, https://spring.io/.
Hoffman, A., 2024. Web application security. O'Reilly Media, Inc. Jan 17.
Pagano, Francesco, et al., 2023. SEBASTiAn: a static and extensible black-box application security testing tool for iOS and Android applications. SoftwareX 23, 101448.
Cifuentes, Cristina, et al., 2023. The role of program analysis in security vulnerability detection: then and now. Comput. Secur. 135, 103463.
Pargaonkar, Shravan., 2023. Advancements in security testing: a comprehensive review of methodologies and emerging trends in software quality engineering. Int. J. Sci. Res. (IJSR) 12, 61–66, 9.
RASP, https://www.gartner.com/en/information-technology/glossary/runtime-application-self-protection-rasp.
Micro Focus, https://en.wikipedia.org/wiki/Micro_Focus.
Contrast Security, https://www.contrastsecurity.com/.
Prevoty, https://www.imperva.com/company/press_releases/imperva-completes-the-acquisition-of-prevoty/.
Shadowd, https://shadowondemandca.com/lander.
OWASP Top 10, https://owasp.org/www-project-top-ten/.
Ding, Yuxin, et al., 2023. Android malware detection method based on bytecode image. J. Ambient Intell. Humani. Comput. 14, 6401–6410, 5.
JDK, Java Develop Kit, https://www.oracle.com/java/.

Lin, J., Adams, B., Hassan, A.E., 2023. On the coordination of vulnerability fixes: an empirical study of practices from 13 CVE numbering authorities. Empir. Softw. Eng. 28 (6), 151.
Xiong, Qiang, et al., 2023. An empirical analysis of vulnerability information disclosure impact on patch R&D of software vendors. J. Intell. Fuzzy Syst. 44, 839–853, 1.
Wong, Ann Yi, et al., 2023. On the security of containers: Threat modeling, attack analysis, and mitigation strategies. Comput. Secur. 128, 103140.
CVE-2022-26201, https://cve.mitre.org/cgi-bin/cvename.cgi?name=2022-26201.
CNVD-2022-42853, https://github.com/ce-automne/CNVD-2022-42853.
Ferreira, Pedro, et al., 2023. Log4j vulnerability. In: International Conference on Information Technology & Systems. Springer International Publishing, Cham.
Wong, Ann Yi, et al., 2023. On the security of containers: Threat modeling, attack analysis, and mitigation strategies. Comput. Secur. 128, 103140.
Peddireddy, Kiran., 2023. Kafka-based Architecture in building data lakes for real-time data streams. Int. J. Comput. Appl. 185, 1–3, 9.
Solr, Covering Apache. "Apache Solr Reference Guide." 2014-10-13. http://mirror.bit.edu.cn/apache/lucene/solr/ref-guide/apache-solr-ref-guide-4.10.pdf (2023).
Lombard, Christophe., 2023. Operating VMware cloud on AWS. VMware Cloud on AWS: Insights on the First VMware Enterprise-Proven SaaS Solution. Apress, Berkeley, CA, pp. 215–284.
CVE-2022-22955, https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2022-22955.
CVE-2022-22957, https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2022-22957.
CNVD-2023-11441, https://vulners.com/cnvd/CNVD-2023-11441.
CNVD-2023-70071, https://nvd.nist.gov/vuln/detail/CVE-2023-50071.
CNVD-2022-79410,https://www.cvedetails.com/vulnerability-list/vendor_id-53/product_id-33138/year-2022/opxss-1/Adobe-Experience-Manager.html.
Siahaan, Mangapul, Wijaya, Ricky, 2024. Performance comparison between laravel and expressJs framework using apache JMeter. J. Inf. Telecomm. Eng. 7, 545–554, 2.
Dstat, https://github.com/dstat-real/dstat.
He, C., Ding, C.H.Q., 2024. A novel classification algorithm for customer churn prediction based on hybrid Ensemble-Fusion model. Sci. Rep. 14, 20179.
He, C., Ding, H.Q., C, 2022. Predicting partner's digital transformation based on artificial intelligence. Appl. Sci. 12, 91.
He, Chenggang, Ding, Chris HQ, 2025. DIAGEMHMM: HMM based on diagonal occupation matrices and EM algorithms for Mendel's law of heredity. Appl. Math. Modell. 139, 115832.